

PHASE 4 — Leave Management & Payroll

PRSECURITY HRMS

IMPORTANT RULES

Output every file 100% completely. Never use '...' or 'rest remains same'.

After each file write "--- FILE COMPLETE ---" then start the next file immediately.

If about to hit output limit, finish current file and write "--- PAUSED: Type CONTINUE ---"

This is Phase 4 of 8. Phases 1-3 done: Auth, Employees, Dashboards, Attendance.

DO NOT rebuild previous files. Only add NEW files or EXTEND where specified.

CONTEXT

Project: PRSECURITY HRMS | Stack: React+Vite+TS+Tailwind+shadcn/ui |

Node+Express+Prisma+PostgreSQL Roles: ADMIN, HR, EMPLOYEE, INTERN | Currency: ₹ | Dates: DD

MMM YYYY

PART A — LEAVE MANAGEMENT

STEP A1 — Update Prisma Schema: Leave Models

Replace placeholder models with full schemas:



prisma

```
model LeaveType {
  id      String   @id @default(uuid())
  name    String
  code    String   @unique
  defaultDays Int
  isPaid  Boolean   @default(true)
  carryForward Boolean @default(false)
  maxCarryForward Int @default(0)
  description String?
  createdAt DateTime @default(now())
  balances LeaveBalance[]
  requests LeaveRequest[]
}
```

```
model LeaveBalance {
  id      String @id @default(uuid())
  userId  String
  leaveTypeId String
  year    Int
  allocated Float
  used    Float @default(0)
  remaining Float
  user    User   @relation(fields: [userId], references: [id])
  leaveType LeaveType @relation(fields: [leaveTypeId], references: [id])
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@unique([userId, leaveTypeId, year])
}
```

```
model LeaveRequest {
  id      String   @id @default(uuid())
  userId  String
  leaveTypeId String
  fromDate DateTime
  toDate   DateTime
  totalDays Float
  reason   String
  status   LeaveStatus @default(PENDING)
  reviewedBy String?
  reviewedAt DateTime?
  reviewerNote String?
  user     User       @relation(fields: [userId], references: [id])
  leaveType LeaveType @relation(fields: [leaveTypeId], references: [id])
  reviewer User?      @relation("LeaveReviewer", fields: [reviewedBy], references: [id])
}
```

```
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
}
```

```
enum LeaveStatus {
  PENDING
  APPROVED
  REJECTED
  CANCELLED
}
```

Also update seed.ts to seed these LeaveTypes:

Casual Leave (CL), 12 days, paid, no carry forward

Sick Leave (SL), 12 days, paid, no carry forward

Earned Leave (EL), 15 days, paid, carry forward max 30

Comp Off (CO), 0 days (granted manually), paid, no carry forward

Maternity Leave (ML), 180 days, paid, no carry forward

Paternity Leave (PL), 15 days, paid, no carry forward

Unpaid Leave (UL), 999 days (unlimited), unpaid, no carry forward

Optional Holiday (OH), 2 days, paid, no carry forward

Seed LeaveBalance rows for admin + hr accounts for current year.

RUN: `npx prisma migrate dev --name add_leave`

STEP A2 — Leave Backend

FILE: backend/src/services/leave.service.ts

`getAllLeaveTypes()`: return all leave types

`createLeaveType(data)`: HR/Admin creates new leave type

`getLeaveBalance(userId, year?)`:

Return all leave balances for user for given year (default current year)

Include leaveType name and code

`getTeamLeaveBalance(year?)`:

Return all users with their leave balances

`applyLeave(userId, data)`:

data: leaveTypeId, fromDate, toDate, reason

Calculate totalDays: count working days between fromDate and toDate (exclude weekends + holidays from HolidayCalendar)

Check remaining balance $\geq$ totalDays (except UL)

Check no overlapping approved/pending leave exists

Create LeaveRequest with status=PENDING

Send notification to HR + Admin: "{user} applied for {leaveType} from {fromDate} to {toDate}"

`getMyLeaveRequests(userId, filters)`:

Paginated list with leaveType, status filters

`getAllLeaveRequests(filters)`:

HR/Admin: all requests with user info, filters: status, leaveTypeId, userId, dateRange

`approveLeave(requestId, reviewerId, note?)`:

Update status=APPROVED, reviewedBy, reviewedAt, reviewerNote
Deduct totalDays from LeaveBalance.used, recalculate remaining
Update Attendance records for those dates to status=ON_LEAVE
Send notification to employee

```
rejectLeave(requestId, reviewerId, note):
```

Update status=REJECTED

Send notification to employee with reason

```
cancelLeave(requestId, userId):
```

Only if status=PENDING and request belongs to userId

Update status=CANCELLED

```
getTeamLeaveCalendar(month, year):
```

Return all approved leave requests where date range overlaps with month/year

Include user name, leave type, dates

FILE: backend/src/controllers/leave.controller.ts

FILE: backend/src/routes/leave.routes.ts

```
□  
  
GET  /api/leaves/types      → getAllLeaveTypes (authenticated)  
POST /api/leaves/types      → createLeaveType (HR/Admin)  
GET  /api/leaves/balance    → getLeaveBalance (own)  
GET  /api/leaves/balance/:userId → getLeaveBalance (HR/Admin)  
GET  /api/leaves/balance/team → getTeamLeaveBalance (HR/Admin)  
POST /api/leaves/apply      → applyLeave (authenticated)  
GET  /api/leaves/my         → getMyLeaveRequests (authenticated)  
GET  /api/leaves            → getAllLeaveRequests (HR/Admin)  
PUT  /api/leaves/:id/approve → approveLeave (HR/Admin)  
PUT  /api/leaves/:id/reject  → rejectLeave (HR/Admin)  
PUT  /api/leaves/:id/cancel  → cancelLeave (own)  
GET  /api/leaves/calendar   → getTeamLeaveCalendar?month=&year= (authenticated)
```

STEP A3 — Leave Cron Job

FILE: backend/src/jobs/leaveReset.job.ts

node-cron job that runs on January 1st every year at 00:01:

For all active users

For each LeaveType: create/reset LeaveBalance rows with allocated=defaultDays, used=0, remaining=defaultDays

For EL (Earned Leave): carry forward min(previous year remaining, maxCarryForward) — add to new allocated

Log: "Leave balances reset for year {year}"

Register this job in app.ts.

STEP A4 — Leave Frontend

FILE: frontend/src/api/leave.api.ts

Functions for all leave endpoints.

FILE: frontend/src/components/leave/LeaveBalanceCards.tsx

Component showing leave balance cards in a grid:

One card per leave type

Card shows: leave type name, code badge, Used/Remaining/Total as horizontal bar, isPaid badge

Compact design to fit multiple cards in a row

FILE: frontend/src/components/leave/LeaveRequestForm.tsx

Form component (used in modal):

Leave Type dropdown (shows balance remaining next to each option)

From Date + To Date date pickers

Total days auto-calculated (shown live, excluding weekends)

Reason textarea

Submit button

FILE: frontend/src/components/leave/LeaveRequestTable.tsx

Table component showing leave requests: Props: `{ requests, onApprove?, onReject?, onCancel?, showEmployee? }`

Columns: Employee (if showEmployee), Leave Type, From, To, Days, Reason (truncated), Status badge,

Applied On, Actions

FILE: frontend/src/pages/leave/LeavePage.tsx

Employee/Intern view:

PageHeader: "My Leave" + "Apply Leave" button

LeaveBalanceCards for current year

"Apply Leave" button opens modal with LeaveRequestForm

Tabs:

All Requests: LeaveRequestTable (own requests), with status filter

Team Calendar: TeamLeaveCalendar view (month view)

FILE: frontend/src/pages/leave/TeamLeavePage.tsx

HR/Admin view:

PageHeader: "Leave Management"

Tabs:

Pending Approvals: LeaveRequestTable with approve/reject buttons. Clicking approve shows confirm modal with optional note. Reject requires note.

All Requests: Full filterable list (by employee, leave type, status, date range)

Leave Calendar: Team leave calendar

Leave Balances: Table of all employees with their leave balances per type

Leave Types: List of leave types + "Add Leave Type" button

FILE: frontend/src/components/leave/TeamLeaveCalendar.tsx

Monthly calendar showing team leave:

Month navigation

Each day cell shows small avatar chips of who is on leave that day

Clicking a day shows popup with list of employees on leave

PART B — PAYROLL

STEP B1 — Update Prisma Schema: Payroll Model



prisma

```

model Payroll {
  id          String   @id @default(uuid())
  userId      String
  month       Int
  year        Int
  basicSalary Float
  hra         Float    @default(0)
  travelAllowance Float @default(0)
  medicalAllowance Float @default(0)
  otherAllowances Float @default(0)
  bonus       Float    @default(0)
  lossOfPay   Float    @default(0)
  otherDeductions Float @default(0)
  pfEmployee  Float    @default(0)
  pfEmployer  Float    @default(0)
  esiEmployee  Float    @default(0)
  esiEmployer  Float    @default(0)
  tds         Float    @default(0)
  grossSalary  Float
  netSalary    Float
  totalDeductions Float
  workingDays  Int
  presentDays  Int
  paymentStatus PaymentStatus @default(PENDING)
  paidAt       DateTime?
  generatedBy  String
  payslipUrl   String?
  user         User      @relation(fields: [userId], references: [id])
  generator    User      @relation("PayrollGenerator", fields: [generatedBy], references: [id])
  createdAt    DateTime  @default(now())
  updatedAt    DateTime  @updatedAt
  @@unique([userId, month, year])
}

```

```

enum PaymentStatus {
  PENDING
  PROCESSING
  PAID
}

```

Run: `npx prisma migrate dev --name add_payroll`

STEP B2 — Payroll Calculator Utility

FILE: `backend/src/utis/payrollCalculator.ts`



typescript

```
interface PayrollInput {  
  basicSalary: number  
  hra: number  
  travelAllowance: number  
  medicalAllowance: number  
  otherAllowances: number  
  bonus: number  
  otherDeductions: number  
  pfEmployeePercent: number  
  pfEmployerPercent: number  
  esiEmployeePercent: number  
  esiEmployerPercent: number  
  tdsPercent: number  
  workingDays: number  
  presentDays: number  
}
```

```
interface PayrollResult {  
  grossSalary: number  
  lossOfPay: number  
  pfEmployee: number  
  pfEmployer: number  
  esiEmployee: number  
  esiEmployer: number  
  tds: number  
  totalDeductions: number  
  netSalary: number  
}
```

```
export function calculatePayroll(input: PayrollInput): PayrollResult
```

Logic:



$dailyRate = basicSalary / workingDays$
 $lossOfPay = dailyRate \times (workingDays - presentDays)$ [only if $presentDays < workingDays$]
 $grossSalary = basicSalary + hra + travelAllowance + medicalAllowance + otherAllowances + bonus - lossOfPay$
 $pfEmployee = basicSalary \times (pfEmployeePercent / 100)$
 $pfEmployer = basicSalary \times (pfEmployerPercent / 100)$
 $esiEmployee = grossSalary \leq 21000 ? grossSalary \times (esiEmployeePercent / 100) : 0$
 $esiEmployer = grossSalary \leq 21000 ? grossSalary \times (esiEmployerPercent / 100) : 0$
 $tds = grossSalary \times (tdsPercent / 100)$
 $totalDeductions = pfEmployee + esiEmployee + tds + otherDeductions$
 $netSalary = grossSalary - totalDeductions$

Round all values to 2 decimal places.

STEP B3 — Payroll Backend

FILE: backend/src/services/payroll.service.ts

`generatePayroll(month, year, generatedByUserId):`

Check payroll not already generated for this month/year

Get all ACTIVE employees with their salaryStructure

For each employee:

Get attendance summary for month/year (workingDays, presentDays)

Calculate payroll using calculatePayroll()

Create Payroll record in DB

Send notification to all employees: "Your payslip for {MonthName Year} is ready"

Return: { generated: count, skipped: count (already existed) }

`getMyPayslips(userId):`

All payroll records for user, ordered by year desc, month desc

`getPayslipDetail(userId, month, year):`

Single payroll record with full details

Check user can only access their own (unless HR/Admin)

`getAllPayroll(filters):`

HR/Admin: all payroll with user info, filter by month/year/status

`markAsPaid(payloadId):`

Update paymentStatus=PAID, paidAt=now

`addBonus(payloadId, bonusAmount):`

Add bonus amount, recalculate gross and net salary, update record

`deletePayroll(payloadId):`

Only if status=PENDING (Admin only)

FILE: backend/src/controllers/payroll.controller.ts

FILE: backend/src/routes/payroll.routes.ts

```
GET /api/payroll/my → getMyPayslips (authenticated)
GET /api/payroll/my/:month/:year → getPayslipDetail (authenticated)
POST /api/payroll/generate → generatePayroll (HR/Admin)
GET /api/payroll → getAllPayroll (HR/Admin)
GET /api/payroll/:userId/:month/:year → getPayslipDetail for user (HR/Admin)
PUT /api/payroll/:id/mark-paid → markAsPaid (HR/Admin)
POST /api/payroll/:id/bonus → addBonus (HR/Admin)
DELETE /api/payroll/:id → deletePayroll (Admin)
```

STEP B4 — Payroll Frontend

FILE: frontend/src/api/payroll.api.ts

FILE: frontend/src/utils/formatCurrency.ts

```
typescript
export const formatINR = (amount: number): string =>
  new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR', maximumFractionDigits: 0 }).format(amount)

export const amountInWords = (amount: number): string => // implement number to Indian words conversion
// e.g. 52340 → "Fifty Two Thousand Three Hundred Forty Only"
```

FILE: frontend/src/components/payroll/PayslipView.tsx

Payslip display component (rendered in browser, same layout used for PDF):

Header: PRSECURITY CONSULTANCY & SERVICES, Surat, India

Employee details grid: Name, Employee ID, Designation, Department, Month-Year, Bank Account (masked)

Two-column table:

Left (Earnings): Basic Salary, HRA, Travel Allowance, Medical Allowance, Other Allowances, Bonus → Gross

Salary total row

Right (Deductions): PF Employee, ESI Employee, TDS, Loss of Pay, Other Deductions → Total Deductions

row

Employer Contributions box: PF Employer, ESI Employer (company pays, not deducted from employee)

Net Pay highlighted box: ₹{netSalary} (bold large font) + amount in words

Attendance: Working Days / Present Days

Footer: Generated on {date} | This is a computer-generated payslip

All amounts formatted with formatINR

FILE: frontend/src/components/payroll/DownloadPayslipButton.tsx

Button that:

On click: fetches payslip data

Uses jsPDF to generate PDF from PayslipView content

Downloads as "Payslip_{EmployeeID}{Month}{Year}.pdf"

FILE: frontend/src/pages/payroll/PayrollPage.tsx

Employee/Intern view:

PageHeader: "My Payslips"

Table of all payslips: Month, Year, Gross Salary, Deductions, Net Salary, Status badge, Download button

Click row → opens PayslipView in modal

FILE: frontend/src/pages/payroll/PayrollManagePage.tsx

HR/Admin view:

PageHeader: "Payroll Management" + "Generate Payroll" button

Generate Payroll button → opens modal: select Month + Year + confirm

Preview table shows all employees with calculated amounts before confirming

After confirm: shows success with count generated

Tab 1 — All Payroll Records:

Filter by Month/Year/Status

Table: Employee, Month/Year, Gross, Net, Status badge, Actions (View Payslip, Add Bonus, Mark as Paid)

Bulk "Mark as Paid" for all PENDING of selected month

Export CSV

Tab 2 — Payroll Summary:

For selected month: total gross payout, total net payout, total PF, total ESI, total TDS

Bar chart (Recharts): net salary distribution across employees

STEP C — Connect to Dashboards

Update backend/src/services/dashboard.service.ts

Admin dashboard: add `payrollDueThisMonth` (count of PENDING payrolls for current month)

Employee dashboard: populate `latestPayslip` with most recent payroll record

Update frontend/src/pages/dashboard/EmployeeDashboardPage.tsx

Connect leave balance cards with real data from `/leaves/balance`

Connect latest payslip card with real data

AFTER ALL FILES ARE COMPLETE

Write this exactly: "  PHASE 4 COMPLETE — Leave Management (apply, approve, reject, balance tracking, team calendar) and Payroll (generation, calculation with PF/ESI/TDS, payslip view, PDF download) are ready. Proceed to Phase 5 for Tasks, Projects & Clients."